GRAVIT

Hackathon Theme: Theme B — Rethinking the System

Problem Domain: Solving the "Broken Communication Stack" in Higher Education

Team: Cucklers (Group No: 45)

1 Problem Statement

The communication infrastructure in higher education institutions is fundamentally broken. Student grievances are trapped in fragmented WhatsApp groups, ignored institutional emails, and performative suggestion boxes — never reaching decision-makers with the urgency or data they require. There is no unified system to aggregate, quantify, or escalate student feedback, and institutions face zero consequence for ignoring complaints entirely. Worse, when administrators do respond, they can mark issues "resolved" without evidence or accountability — and students have no mechanism to challenge fake fixes. This systemic failure creates a cycle of complaint fatigue, institutional inertia, and eroding trust that no existing tool addresses because the problem is not communication — it is the complete absence of enforced accountability.

2 Proposed Idea

Gravit is a centralized, real-time web platform that converts isolated student grievances into structured, publicly tracked, and automatically escalated institutional data. It is built around three core mechanisms that no existing platform offers:

1) Velocity-Weighted Urgency Scoring — An exponential time-decay algorithm that measures how fast upvotes are growing, not just how many exist. This detects emerging crises in real time (50 upvotes in 2 hours triggers faster than 200 over a month).

2) Dead Man's Switch (DMS) — An automatic hierarchical escalation system. If an escalated issue receives no admin response within a configured time window, it auto-escalates to the next level in the institutional hierarchy (HoD → Dean → VC → Public Transparency Report). Silence becomes progressively more expensive.

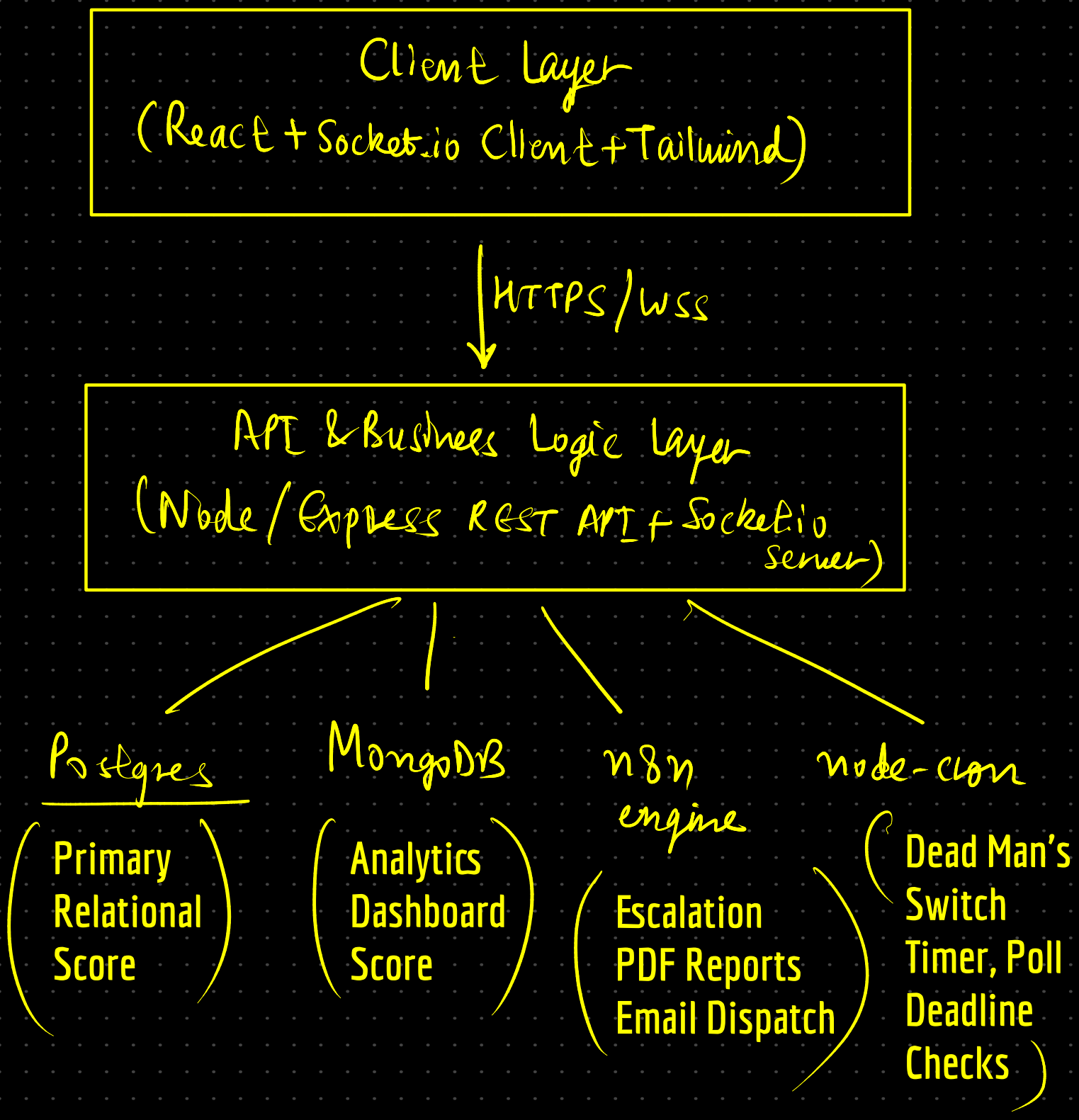
3) Resolution Verification Loop — When an admin marks an issue "Resolved," students receive a 48-hour verification poll. If >50% reject the claimed fix, the issue automatically re-escalates at the next hierarchy level with a "Resolution Rejected" badge. Administrators cannot performatively close tickets.

These three mechanisms work together to create a system where complaints cannot be lost, silence is punished, and fixes must be real.

3 Solution Architecture

3.1 High-Level Architecture

Gravit follows a decoupled, event-driven architecture with five distinct subsystems:



3.2 Tech Stacks

Frontend: React.js + Socket.io-client + Tailwind CSS

Role in Architecture	Details
Component-based UI	Each core feature (post feed, threaded comments, urgency meter, DMS countdown timer, verification poll) is an isolated React component, enabling parallel development across team members.
Real-time rendering	Socket.io-client subscribes to post-specific rooms. When any user upvotes, the urgency score recalculates server-side and pushes a `urgency:update` event — the React urgency meter updates instantly without page reload.
Optimistic UI	Upvotes and poll votes render immediately on the client (optimistic state), then reconcile with server confirmation within milliseconds. This makes the platform feel instant.
Live DMS Timer	A countdown component receives the `response_deadline` from the API and ticks down in real time. When it hits zero, a `dms:triggered` Socket event updates the badge to "⏰ Escalated to Dean."
Verification Poll UI	When an admin claims resolution, all subscribed clients receive `verification:start` and render an inline Yes/No poll with a live-updating tally bar.

Backend: Node.js + Express.js

Role in Architecture	Details
REST API	All CRUD operations — posts, comments, upvotes, admin responses, verification votes — exposed via Express routes with JWT-authenticated middleware.
Socket.io Server	Runs alongside Express on the same Node process. Manages 4 namespaces: `/posts` (upvote + urgency sync), `/status` (state transitions + DMS), `/presence` (active user count), `/verify` (resolution poll events). Uses room-based broadcasting (`post:{id}`, `channel:{name}`, `institution:{id}`) so events reach only relevant clients.
State Machine Engine	Enforces the 6-state post lifecycle (`open → trending → escalated → pending_verification → resolved / resolution_rejected`). All transitions are validated server-side — the client cannot force a state change. Transitions are irreversible forward.
Velocity Scoring Engine	On every upvote, the API recalculates the post's urgency score using the exponential time-decay formula. If the score crosses T_1 (trending) or T_2 (escalation), the state machine transitions automatically and fires the n8n webhook.
RBAC Middleware	Three roles enforced at the route level: Student (post, vote, verify), Moderator (merge, flag), Admin (respond, view dashboard). Admins **cannot** access user identities or delete posts.
Pseudonymous Identity	On registration, a persistent pseudonym (`Anon-7F3A`) is generated via one-way hash of the user's internal UUID + institution-scoped salt. The real-identity ↔ pseudonym mapping is stored in an encrypted column, inaccessible to application-level queries.

Primary Database: PostgreSQL

Role in Architecture	Details
Transactional integrity	ACID compliance is critical for: upvote deduplication (composite PK on `user_id, target_id`), state transitions (only valid forward transitions), and verification vote counting (exactly one vote per user per poll).
Threaded comments	Materialized Path pattern — each comment stores its full ancestry as a path string (e.g., `root/a1b2/c3d4`). Fetching an entire thread is a single query: `SELECT * FROM comments WHERE post_id = $1 ORDER BY path`. Insertion is O(1).
Escalation hierarchy	The `escalation_hierarchy` table maps each channel × level to a specific contact email and response window. The DMS cron job joins against this table to determine who to notify next and how long they have.
Velocity scoring support	Every upvote has a `created_at` timestamp. The urgency score query reads all vote timestamps and applies the decay function. PostgreSQL's `TIMESTAMPTZ` type handles timezone-safe calculations natively.

Core tables: `institutions`, `users`, `posts` (with `urgency_score`, `current_escalation_level`, `escalated_at`, `last_admin_response_at`), `comments`, `upvotes`, `escalations`, `escalation_hierarchy`, `resolution_verifications`, `verification_votes`.

Analytics Store: MongoDB

Role in Architecture	Details
Denormalized dashboard data	The admin dashboard needs pre-aggregated data (urgency heatmaps, resolution time distributions, DMS trigger frequency). Writing these aggregations to MongoDB's schema-flexible documents avoids expensive real-time joins on PostgreSQL.
Time-series urgency data	Urgency scores are logged periodically to MongoDB, enabling "urgency score over time" charts that show how fast an issue gained traction — useful for the admin dashboard's trending visualization.
Non-blocking writes	Analytics writes don't interfere with the primary transactional workload on PostgreSQL. The API fires and forgets to MongoDB after each state change.

Workflow Automation: n8n (self-hosted)

Role in Architecture	Details
Decoupled escalation logic	PDF generation, email templating, and SMTP dispatch are NOT in the core API. They live entirely in n8n workflows, making the escalation pipeline independently iterable without touching backend code.
Three-branch escalation	A single webhook endpoint receives all escalation triggers. An n8n **Switch node** branches by `trigger_type`: (1) `threshold` — standard urgency-triggered report, (2) `dead_mans_switch` — report with non-response timeline and "Ignored at X levels" header, (3) `resolution_rejected` — report with admin's claimed fix and student rejection vote tally.
PDF generation	n8n's HTML-to-PDF node (or a Function node with `pdfkit`) compiles the thread data into a branded, formal PDF: issue title, urgency score, top comments, engagement metrics, escalation history, and a direct link to the post.
Email dispatch	n8n's SMTP node (or SendGrid/Mailgun node) sends the PDF-attached email to the recipient determined by `escalation_level` × `escalation_hierarchy` table. Callback to the API logs the dispatch in the `escalations` table.
Why n8n over custom code	Building 3 PDF templates + email dispatch + retry logic from scratch = 3-4 days. Doing it in n8n's visual builder = **3-4 hours**. This is the critical hackathon force multiplier.

Scheduling: node-cron

Role in Architecture	Details
Dead Man's Switch monitor	Runs **every hour**. Queries all posts in `escalated` or `resolution_rejected` status where `last_admin_response_at IS NULL` and the response window has elapsed. For each match, fires the n8n webhook at the next hierarchy level.
Verification poll enforcer	Runs **every 15 minutes**. Queries `resolution_verifications` where `outcome = 'pending'` and `deadline < NOW()`. Tallies votes and: if >50% reject → transitions post to `resolution_rejected` and triggers re-escalation; if >50% confirm → transitions to `resolved`.
Why node-cron	Lightweight, zero-dependency, runs in the same Node.js process. No need for external schedulers (Celery, Bull) for a hackathon prototype.

4 How the USPs Work in the Architecture

4.1 Velocity-Weighted Urgency Scoring

$$S(t) = \sum_{i=1}^{n} e^{-\lambda \cdot (t_{now} - t_i)}$$

- λ = 0.05/hour (configurable). Recent upvotes contribute ~1.0 weight, 24h-old votes ~0.3, week-old votes ~0.0.

- Calculated in the Node.js API on every upvote event by querying created_at timestamps from the upvotes table in PostgreSQL.

- The updated score is written to posts.urgency_score and broadcast to all subscribed clients via Socket.io (urgency:update event).

- The React frontend renders this as a live color-coded urgency meter (green → yellow → orange → red).

- When the score crosses T_2, the state machine transitions the post to escalated and fires the n8n webhook.

Scenario	Raw Count	Urgency Score	Result
50 upvotes in 2 hours	50	~45.2	Escalated
200 upvotes over 30 days	200	~12.8	Trending only
100 upvotes, 80 in last 6h	100	~68.4	Escalated

4.2 Dead Man's Switch (DMS)

Flow through the stack:

1) Post reaches escalated status → posts.escalated_at is set (PostgreSQL).

2) node-cron (hourly) joins posts with escalation_hierarchy to find posts where the response window has elapsed with no admin response.

3) For each match, the cron job increments posts.current_escalation_level, fires the n8n webhook with trigger_type: 'dead_mans_switch'.

4) n8n Switch node routes to the DMS branch → generates a PDF with non-response timeline + "Ignored at X levels" header → emails the next-level recipient.

5) Socket.io broadcasts dms:triggered to all clients → React updates the post badge: "Escalated to Dean — HoD did not respond in 72 hours."

```
Level 1: HoD ——— 72h silence ———► Level 2: Dean
Level 2: Dean —— 120h silence ——► Level 3: VC
Level 3: VC ———— 168h silence ——► Level 4: Public Transparency Report
```

4.3 Resolution Verification Loop

Flow through the stack:

1) Admin clicks "Mark Resolved" and provides a resolution description → Express API creates a resolution_verifications record with a 48h deadline (PostgreSQL).
Post status → pending_verification.

2) Socket.io emits verification:start to all clients in the post's room → React renders an inline poll: "Was this actually resolved? Yes / No"

3) Students cast votes → stored in verification_votes (one per user per poll, PostgreSQL).
Each vote broadcasts a verification:vote event → React updates the live tally bar.

4) node-cron (every 15 min) checks expired polls:
   - >50% confirm → resolved. Department credited on leaderboard.
   - >50% reject → resolution_rejected. Post auto re-escalates at next hierarchy level.
   - n8n fires a report: "Resolution rejected by X% of N verified students."

5 State Machine — Post Lifecycle

State	Trigger	What Happens
Open	Post created	Collecting upvotes. Urgency score computing in real time.
Trending	Urgency score ≥ T_1	Highlighted in feeds. Urgency meter turns yellow/orange.
Escalated	Urgency score ≥ T_2	n8n generates PDF → emails HoD. **DMS timer starts.**
Pending Verification	Admin responds	48h student poll. Post is **not** closed yet.
Resolved	>50% students confirm	Verified resolution. Leaderboard credit. Archived.
Resolution Rejected	>50% students reject	Auto re-escalates at next hierarchy level with rejection proof.

6 Sub-Channels & Escalation Mapping

Channel	Scope	Level 1	Level 2	Level 3	Level 4
c/Academia	Curriculum, faculty, grading	HoD	Dean	VC	Public
c/Bureaucracy	Fees, documents, registration	Registrar	Admin Head	VC	Public
c/Infrastructure	Labs, hostels, safety	Estate Officer	Dean	VC	Public
c/Placement	Placement cell, companies	TPO	Dean	VC	Public

Each channel maintains independent urgency thresholds and a full DMS hierarchy chain configured in the escalation_hierarchy table.

7 Auth & Identity System

Feature	Implementation
Verification	.edu email domain whitelisting + OAuth 2.0 / JWT token verification. Only verified institutional members can access the platform.
Pseudonymity	Persistent pseudonym (`Anon-7F3A`) generated via one-way hash (user UUID + institution salt). Real identity encrypted, inaccessible to admins/students.
Deanonymization	Multi-party authorization only (legal subpoena / verified abuse report). Never unilateral.
RBAC	Student: post, upvote, comment, verify. Moderator: flag, merge, tag. Admin: respond, view analytics. Admins **cannot** see identities or delete posts.

8 Auth & Identity System

Department Leaderboard

Leaderboard is powered by verified data — resolutions must pass student verification to count:

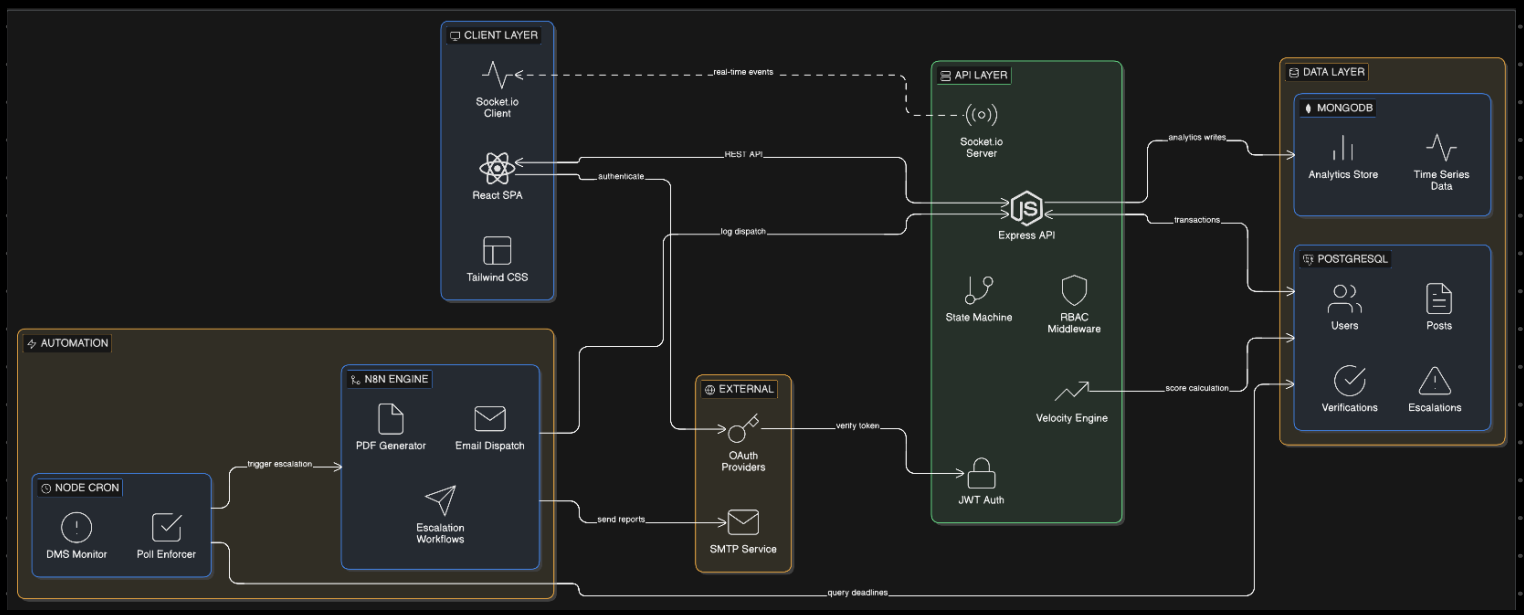
- Average verified resolution time — only student-confirmed resolutions count.
- First-response time — DMS triggers logged as "No Response" penalty.
- Verification success rate — high rejection rate = flagged as performative.
- DMS trigger count — public "Non-Responses" metric.

- Departments cannot game the leaderboard by closing tickets without real fixes.

Stakeholder Impact

Stakeholder	Before Gravit	After Gravit
Students	Complaints vanish. Fake fixes unchallenged.	Public, velocity-prioritized, auto-escalated. Students verify fixes.
HoDs	No urgency to respond.	Visible DMS countdown. Respond or get escalated to Dean.
Dean/VC	Hears about issues too late via informal channels.	Receives formal reports only when lower levels failed — with evidence.
Institution	Accreditation gaps. Social media reputation risk.	Auditable, verified resolution data. Internal resolution before public exposure.

9 Gravit's Architecture



10 Why Gravit Wins

1) Verified voices, protected identities. .edu verification ensures authenticity; pseudonymous posting ensures safety.

2) Velocity-scored, not just counted. Detects crises by momentum, not volume. 50 upvotes in 2 hours > 200 over a month.

3) Silence is punished. Dead Man's Switch auto-escalates up the hierarchy. Ignoring makes it worse, not quieter.

4) Students verify the fix. Fake resolutions re-escalate with proof. "Resolved" on Gravit means students confirmed it.

5) Performance is public and ungameable. Leaderboards track verified metrics, DMS triggers, and rejection rates.